

CSE152B - Spring 2022: Homework 4

Q1: UNet for Image Segmentation

ALL the scripts here should be run from `./Segmentation`

1. **Check the codes in `Segmentation`** . In this homework, we have provided the training script, the evaluation code, 3 versions of the UNet structures and some useful functions. You will be asked to test out different variations of network structures and report their performance on the test set. The provided codes include:
 - A. `test.py` : The file for evaluation.
 - B. `dataLoader.py` : The file to load the data for training and testing.
 - C. `model.py` : The file for models. The residual block (`ResBlock`) and the code to load pretrained weights of `resnet18` are given as `loadPretrainedWeight` . The basic encoder and decoder (`encoder` and `decoder`), as well as the ones with dilation/SPP (`encoderDilation` and `decoderDilation`) are also given.
 - D. `colormap.mat` : The color map used to visualize segmentation results.
 - E. `utils.py` : The file for some useful functions. The `computeAccuracy` function computes the unnormalized confusion matrix of each batch of labels. The `save_label` function turns the label into an image using the given color map and saves the image at the assigned location. Also see `test.py` for how these two functions are being used.
 - F. `train.py` : The file of the training script.

The network structures. There are 3 versions of UNet structures. In all three versions, the `resnet18` structure before average pooling and fully connected layer is be used as the building block for encoder.

1. **Basic UNet** : Please see `encoder` and `decoder` class in `model.py` . The `encoder` comes from `resnet18` and the decoder consists of transpose convolutional layers and bilinear interpolation layers so that the final output will be of the same size as the image. Skip links are added to help the network recover more details.
2. **UNet with dilation** : We modify the encoder to a dilated `resnet18` as described in Section 2 of [1]. We set the stride of the last 4 residual blocks to be 1 so that the highest level feature maps will be 4×4 times larger. To increase the receptive field, we set the dilation of residual blocks that are fourth and third from the end to be 2, while the dilation of the residual blocks that are first and second from the end are set to 4. The new encoder and decoder are implemented under class `encoderDilation` and `decoderDilation` .

3. **UNet with dilation and pyramid pooling** : Based on the encoder-decoder structure with dilation, pyramid pooling layer is added after the last residual block of encoder. The pyramid pooling layer has been implemented following [2]. Notice that after adding the pyramid layer, the number of channels of the output feature to the first transpose convolutional layer will change from 512 to 1024. The new encoder and decoder are implemented under classes `encoderDilation` and `decoderDilation` (`isSpp = True`), respectively.

How the networks are trained. The networks are trained using 1464 images from the training set of PASCAL VOC 2012.

For example to train with dilation and SPP the following script is used:

```
python train.py --isDilation --isSpp --nepoch 200 --isPretrained True --  
experiment checkpoints/unet_original_zq
```

2. Answer the following questions:

- A. Describe the loss function that is used to train the semantic segmentation network.

[10 points]

- B. Evaluate the trained models of `Basic UNet`, `UNet with dilation`, `UNet with dilation and pyramid pooling` provided as checkpoints using the following commands. Draw a table to summarize quantitative performances of the 3 variations of the UNet structure. The table should include the mean IoU of 21 categories of objects and their average mean IoU. The testing mean IoU of 21 categories of object will be saved to `.npy` files following the last line of `test.py`.

You can make changes to `test.py` if necessary. **[10 points]** :

- a. `Basic UNet` : You should load the pre-trained model with `--modelRoot`.

```
python test.py --experiment checkpoints/unet_original_zq --modelRoot  
~/public/unet_checkpoints/unet_original_zq --epochId 181
```

2. `UNet with dilation` :

```
python test.py --experiment checkpoints/unet_original_zq_dilation --isDilation  
--modelRoot ~/public/unet_checkpoints/unet_original_zq --epochId 180
```

3. `UNet with dilation and pyramid pooling` :

```
python
```

```
python test.py --experiment checkpoints/unet_original_zq_spp --isSpp -  
--modelRoot ~/public/unet_checkpoints/unet_original_zq --epochId 180
```

- E. Make a figure for qualitative comparisons of the 3 methods, shown on 4 different input images. Please show the segmentation results for the same image but different networks so the differences can be compared. Briefly describe the results you obtain and any observations. **[10 points]**
- F. Explain your observations in terms of: (i) what choices helped improve the accuracy and (ii) what other steps could have been tried to further improve accuracy? **[10 points]**

points]

The methods in calculating loss achieve the calculation in the test.py file through using the `torch.mean(pred * labelBatch)` method. The `pred` variable can be decrived as the probability of the predictions mapped from the decoder block. The inputs here, going into `pred` would be the image batch of `xs` where they are convolutions of inptu images. its all being tested or checked on the `labelBatch` which is ground truth, or labels for the images. Then the final loss is the average (mean) as the method above describes.

```
In [6]: import numpy as np
import pandas as pd

basic_unet = np.load("Segmentation/checkpoints/unet_original_zq/accuracy_181.npy")
dilated_unet = np.load("Segmentation/checkpoints/unet_original_zq_dilation_dilation")
spp_unet = np.load("Segmentation/checkpoints/unet_original_zq_spp_spp/accuracy_180.npy")

# Category labels from the data set used in training (stated above)
categories = [
    "background", "aeroplane", "bicycle", "bird", "boat", "bottle", "bus",
    "car", "cat", "chair", "cow", "diningtable", "dog", "horse", "motorbike",
    "person", "pottedplant", "sheep", "sofa", "train", "tvmonitor"
]

# Combine into a DataFrame it worked yay
df = pd.DataFrame({
    "Category": categories,
    "Basic UNet": basic_unet,
    "UNet + Dilation": dilated_unet,
    "UNet + Dilation + PPM": spp_unet #kept the same as example from piazza btw
})

df.loc["Mean IoU"] = ["Average"] + list(df.iloc[:, 1:].mean())

print(df.to_string(index=False))
```

Category	Basic UNet	UNet + Dilation	UNet + Dilation + PPM
background	87.937775	87.548241	88.064217
aeroplane	68.100998	67.702469	68.381874
bicycle	41.775528	33.916882	31.369686
bird	58.224567	58.315475	66.249680
boat	47.692921	43.019222	49.024666
bottle	49.854370	46.250458	56.112373
bus	67.648010	68.747147	74.385025
car	66.447868	63.525002	70.398499
cat	66.766716	64.168304	74.010246
chair	16.525263	14.814768	18.243742
cow	50.786549	44.638954	55.562820
diningtable	38.596081	36.534733	41.813545
dog	57.861450	55.015850	65.411850
horse	46.909828	44.806374	56.418625
motorbike	59.433586	59.570599	65.893600
person	74.386322	73.151123	73.465225
pottedplant	37.098392	33.887611	34.630505
sheep	54.980553	55.480431	69.333923
sofa	33.838276	30.195126	32.144684
train	57.628082	59.478630	64.387222
tvmonitor	49.059135	50.448864	53.914658
Average	53.883442	51.962677	57.581741

```
In [21]: import os
import matplotlib.pyplot as plt
from matplotlib.image import imread

model_paths = {
    "Basic UNet": "./Segmentation/checkpoints/unet_original_zq",
    "UNet + Dilation": "./Segmentation/checkpoints/unet_original_zq_dilation_dilati",
    "UNet + Dilation + PPM": "./Segmentation/checkpoints/unet_original_zq_spp_spp"
}

image_indices = ['2007_000346', '2008_007811', '2007_000663', '2007_000862']

fig, axes = plt.subplots(len(image_indices), len(model_paths) + 1, figsize=(18, 12))

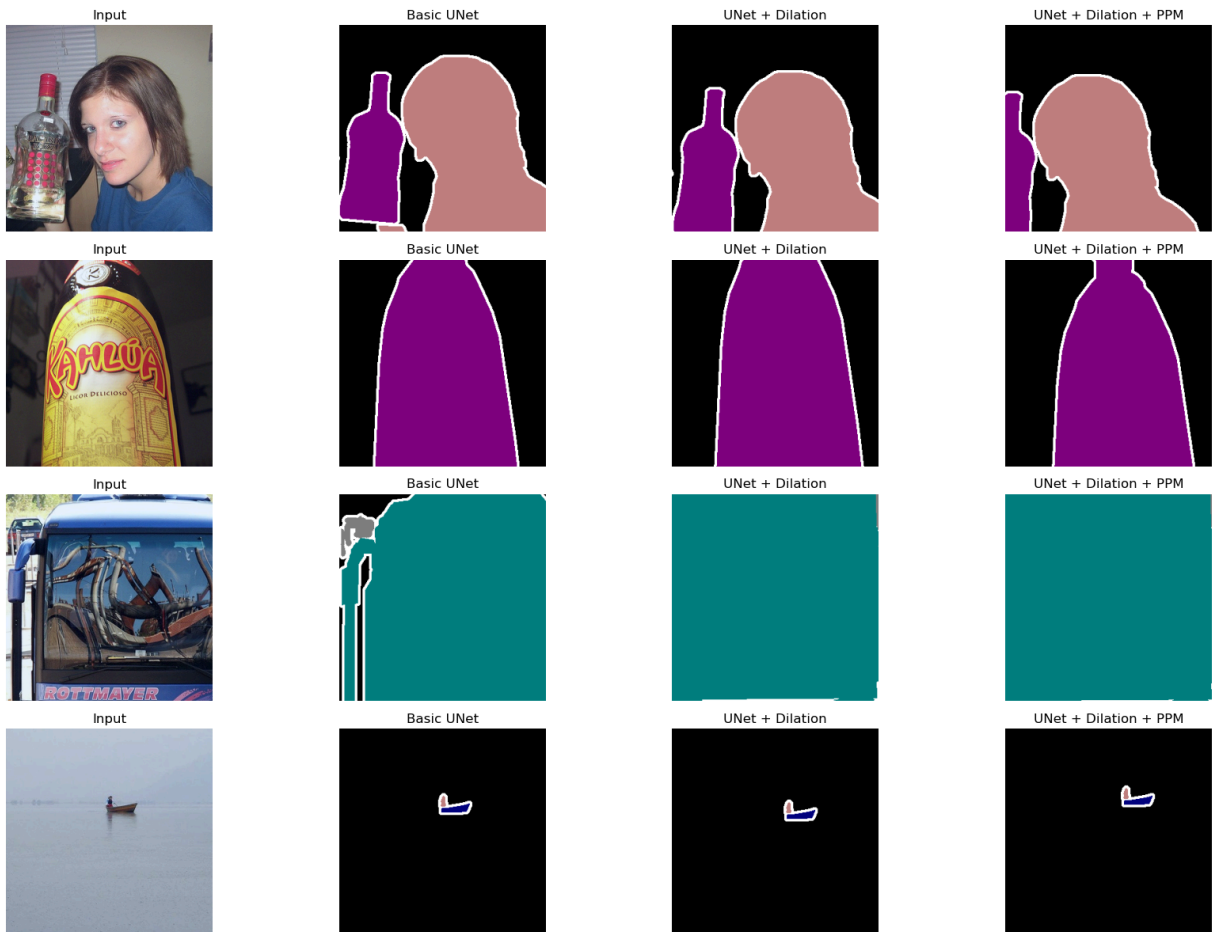
for row_idx, idx in enumerate(image_indices):
    input_path = os.path.join(list(model_paths.values())[0], f"images_{idx}.png")
    input_img = imread(input_path)

    axes[row_idx, 0].imshow(input_img)
    axes[row_idx, 0].set_title("Input", fontsize=12)
    axes[row_idx, 0].axis("off")

    for col_idx, (model_name, model_path) in enumerate(model_paths.items(), start=1):
        pred_path = os.path.join(model_path, f"labelGt_{idx}.png")
        pred_img = imread(pred_path)

        axes[row_idx, col_idx].imshow(pred_img)
        axes[row_idx, col_idx].set_title(model_name, fontsize=12)
        axes[row_idx, col_idx].axis("off")
```

```
plt.tight_layout()
plt.show()
```



Observing the results from above, the dilation seems to change the field of view, while all models are effective at distinguishing and labeling objects from a mid to far distance. Based on the third image, to classify up close objects I would use the basic UNet as it seems to have better distinction for details in the scope we are looking for. To improve accuracy it might be better to have discernable margins in the middle of the image.

Q2: SSD [3] Object Detection

1. **Check the codes in Detection** . The codes are modified from <https://github.com/amdegroot/ssd.pytorch> . Run `eval.py` code to get the object detection average precision (AP) on the PASCAL VOC 2012 dataset. The model is already trained on the PASCAL VOC 2012 object detection dataset. Draw a table in your report summarizing the AP of all 20 object categories and their mean. **[10 points]**

Results:

Class	AP (%)
-----	-----
aeroplane	90.7

bicycle	90.6	
bird	89.9	
boat	86.8	
bottle	74.6	
bus	90.4	
car	86.5	
cat	90.9	
chair	84.7	
cow	90.3	
dining table	88.6	
dog	90.9	
horse	90.9	
motorbike	90.8	
person	88.0	
potted plant	79.6	
sheep	87.6	
sofa	90.7	
train	90.8	
tvmonitor	90.6	
Mean AP	**88.18**	

2. Answer the following questions:

- A. Briefly explain how average precision is computed for PASCAL VOC 2012 dataset. Please check the code (`eval.py` : Line 163 – 191) since there are different ways to compute average precision. **[10 points]**
- B. Explain how SSD can be much faster compared to Faster RCNN [4]? **[10 points]**
- C. Usually the number of negative bounding boxes (boxes without any object) is much larger than the number of positive bounding boxes. Explain how this imbalance is handled in SSD and Faster RCNN, respectively. **[10 points]**

The average precision is computed using the VOC 2007 metric as the flag is set to true by default. It then loops through each class and computes the metric on each, using a threshold of 0.5 which requires more than 50 percent IoU. The "11 point interpolation" metric takes, you guessed it, 11 points at different recall levels (intervals of .1) and averages their precision scores.

SSD can be faster because its fully convolutional, and because it guesses the bounding region in a single pass without doing extra stages and layers.

In SSD, hard negative mining is used in the model, which means it will maintain the imbalance, using the hardest (furthest from negative prediction toward positive) to ensure more precision in the classification. The RCNN, much more quickly uses mini-batch sampling and aims at getting an even split between positives and negatives, which attempts to balance the calculations and calibrations between positives and negatives.

3. Randomly pick up some images from the PASCAL VOC 2012 dataset and some from other sources. Visualize the bounding box prediction results and include a figure in your report. You can use the code in folder **demo** for visualization. **[10 points]**

```
In [3]: import os
import sys
module_path = os.path.abspath(os.path.join('Detection/'))
if module_path not in sys.path:
    sys.path.append(module_path)

import torch
import torch.nn as nn
import torch.backends.cudnn as cudnn
from torch.autograd import Variable
import numpy as np
import cv2
if torch.cuda.is_available():
    torch.set_default_tensor_type('torch.cuda.FloatTensor')

from ssd import build_ssd
from data import VOCDetection, VOCAnnotationTransform
VOC_ROOT = os.path.expanduser('~/.public/VOCdevkit/')

testset = VOCDetection(VOC_ROOT, [('2012', 'val')], None, VOCAnnotationTransform())
```

```
/opt/conda/lib/python3.11/site-packages/torch/__init__.py:696: UserWarning: torch.set_default_tensor_type() is deprecated as of PyTorch 2.1, please use torch.set_default_dtype() and torch.set_default_device() as alternatives. (Triggered internally at
../torch/csrc/tensor/python_tensor.cpp:451.)
  _C._set_default_tensor_type(t)
```

```
In [11]: import random

img_id = random.randint(0, len(testset) - 1)
```

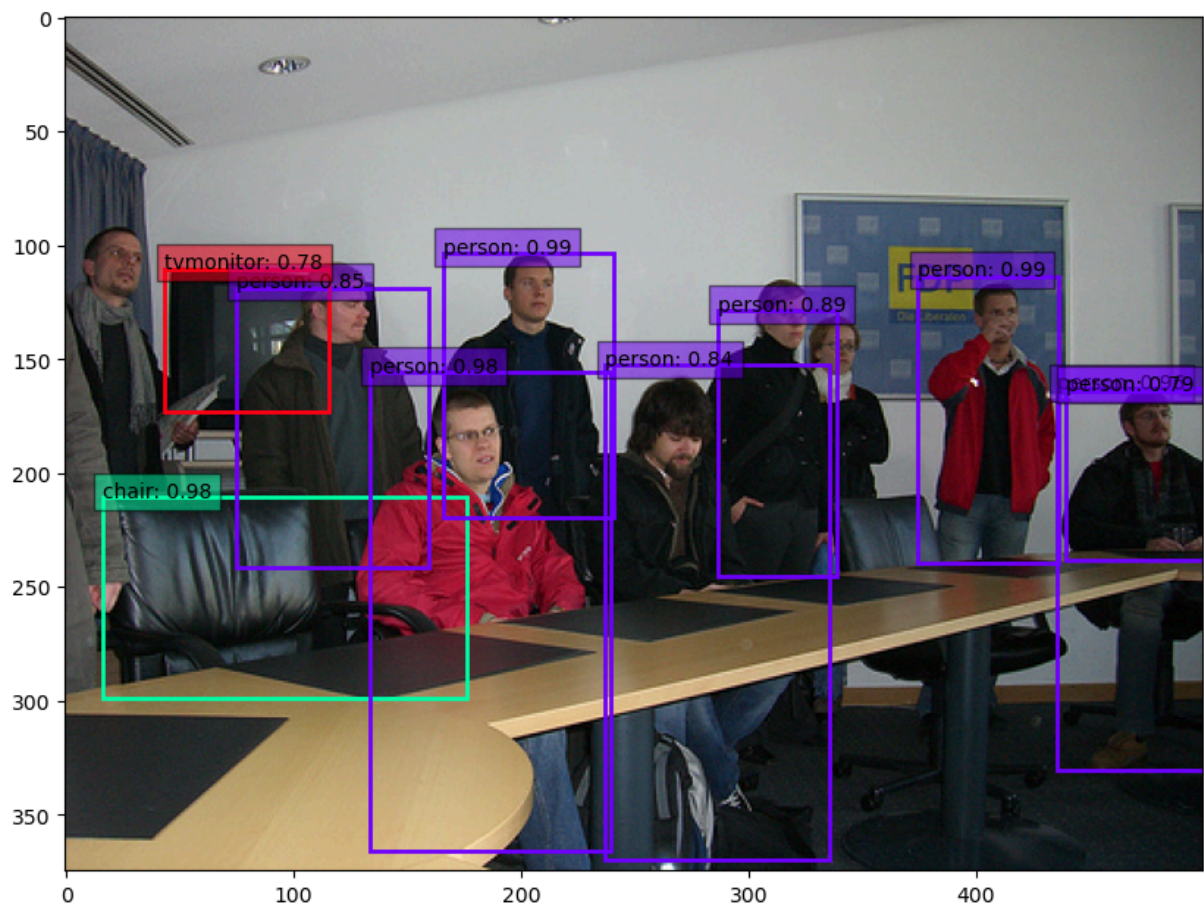
```
In [12]: image = testset.pull_image(img_id) # raw BGR image
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
x = cv2.resize(image, (300, 300)).astype(np.float32)
x -= (104.0, 117.0, 123.0) # mean subtraction
x = x[:, :, ::-1].copy() # BGR to RGB (or RGB to BGR if needed)
x = torch.from_numpy(x).permute(2, 0, 1)
xx = Variable(x.unsqueeze(0))
if torch.cuda.is_available():
    xx = xx.cuda()
net = build_ssd('test', 300, 21) # initialize SSD
net.load_weights(os.path.expanduser('~/.public/detection/weights/VOC.pth'))
y = net(xx) # Run SSD model
detections = y.data
scale = torch.Tensor(rgb_image.shape[1::-1]).repeat(2)
```

```
512
256
256
256
Loading weights into state dict...
Finished!
```

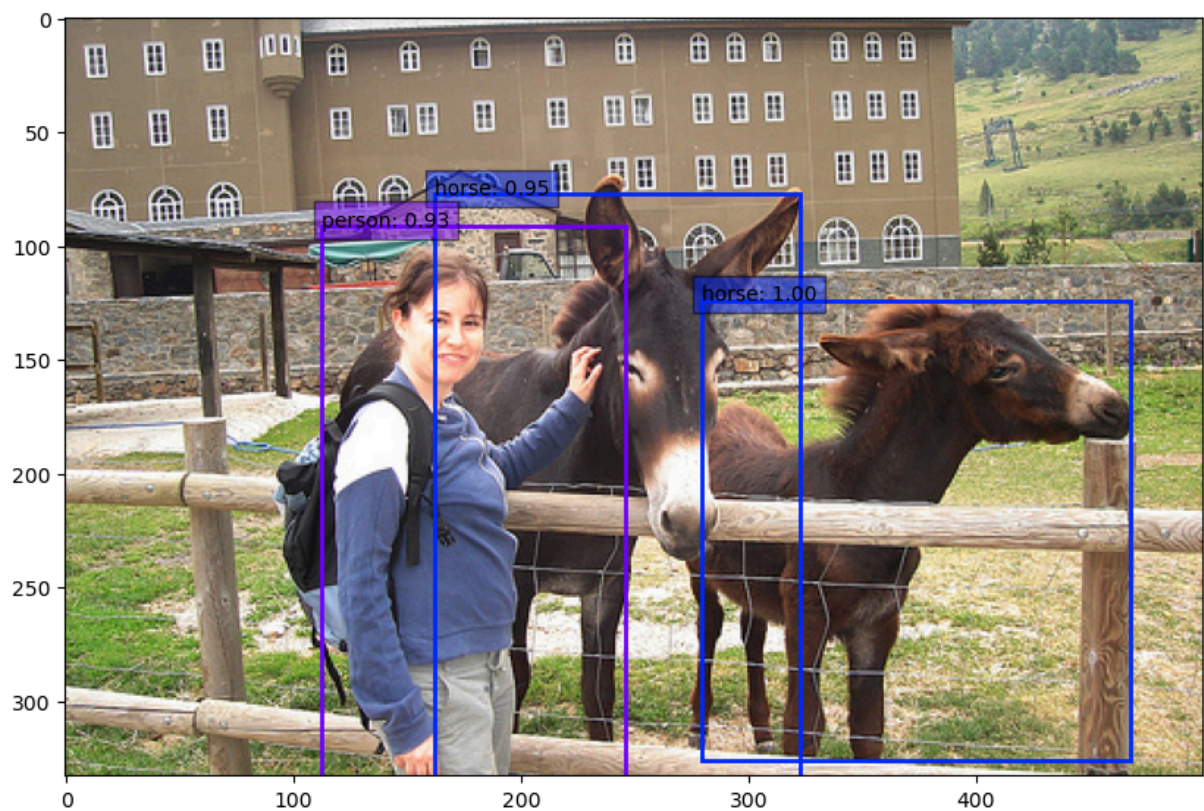
```
In [8]: from data import VOC_CLASSES as labels
import matplotlib.pyplot as plt
from matplotlib.image import imread
top_k=10

plt.figure(figsize=(10,10))
colors = plt.cm.hsv(np.linspace(0, 1, 21)).tolist()
plt.imshow(rgb_image) # plot the image for matplotlib
currentAxis = plt.gca()

detections = y.data
# scale each detection back up to the image
scale = torch.Tensor(rgb_image.shape[1::-1]).repeat(2)
for i in range(detections.size(1)):
    j = 0
    while detections[0,i,j,0] >= 0.6:
        score = detections[0,i,j,0]
        label_name = labels[i-1]
        display_txt = '%s: %.2f'%(label_name, score)
        pt = (detections[0,i,j,1:]*scale).cpu().numpy()
        coords = (pt[0], pt[1]), pt[2]-pt[0]+1, pt[3]-pt[1]+1
        color = colors[i]
        currentAxis.add_patch(plt.Rectangle(*coords, fill=False, edgecolor=color, l
        currentAxis.text(pt[0], pt[1], display_txt, bbox={'facecolor':color, 'alpha
        j+=1
```

In [13]:



```

In [10]: from sklearn.datasets import load_sample_image

china = load_sample_image("china.jpg") # shape (427, 640, 3)
flower = load_sample_image("flower.jpg")
image_list = [china, flower]

net = build_ssd('test', 300, 21) # 'test' phase, input size 300, 21 classes
net.load_weights(os.path.expanduser('~/.public/detection/weights/VOC.pth'))
net.eval()
if torch.cuda.is_available():
    net = net.cuda()

for idx, image in enumerate(image_list):
    image = image[:, :, ::-1] # convert RGB to BGR for OpenCV compatibility
    rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Preprocessing
    x = cv2.resize(image, (300, 300)).astype(np.float32)
    x -= (104.0, 117.0, 123.0) # mean subtraction
    x = x[:, :, ::-1].copy() # BGR to RGB
    x = torch.from_numpy(x).permute(2, 0, 1) # HWC to CHW
    xx = Variable(x.unsqueeze(0)) # Add batch dimension
    if torch.cuda.is_available():
        xx = xx.cuda()

    with torch.no_grad():
        y = net(xx)

    plt.figure(figsize=(10, 10))
    colors = plt.cm.hsv(np.linspace(0, 1, 21)).tolist()
    plt.imshow(rgb_image)
    currentAxis = plt.gca()
    detections = y.data
    scale = torch.Tensor(rgb_image.shape[1::-1]).repeat(2)

    for i in range(detections.size(1)):
        j = 0
        while detections[0, i, j, 0] >= 0.6:
            score = detections[0, i, j, 0]
            label_name = labels[i - 1]
            display_txt = '%s: %.2f' % (label_name, score)
            pt = (detections[0, i, j, 1:] * scale).cpu().numpy()
            coords = (pt[0], pt[1]), pt[2] - pt[0] + 1, pt[3] - pt[1] + 1
            color = colors[i]
            currentAxis.add_patch(
                plt.Rectangle(*coords, fill=False, edgecolor=color, linewidth=2))
            currentAxis.text(pt[0], pt[1], display_txt,
                             bbox={'facecolor': color, 'alpha': 0.5})
            j += 1
    plt.title(f"Sample Image {idx+1}")
    plt.show()

```

512
256
256
256
Loading weights into state dict...
Finished!

Sample Image 1





References

1. Yu, Fisher, and Vladlen Koltun. "Multi-scale context aggregation by dilated convolutions." arXiv preprint arXiv:1511.07122 (2015).
2. Zhao, Hengshuang, et al. "Pyramid scene parsing network." Proceedings of the IEEE conference on computer vision and pattern recognition. 2017.
3. Liu, Wei, et al. "SSD: Single shot multibox detector." European conference on computer vision. Springer, Cham, 2016.
4. Ren, Shaoqing, et al. "Faster R-CNN: Towards real-time object detection with region proposal networks." Advances in neural information processing systems. 2015.